LeetCode HOT 100

PART 13

Heap

堆

74. 数组中的第K个最大元素 (No.215) MEDIUM

给定整数数组 `nums` 和整数 `k`，请返回数组中第 `k` 个最大的元素。

请注意，你需要找的是数组排序后的第 `k` 个最大的元素，而不是第 `k` 个不同的元素。

你必须设计并实现时间复杂度为 `O(n)` 的算法解决此问题。

示例 1:

> **输入：** `[3,2,1,5,6,4]`, `k = 2`
> **输出：** 5

示例 2:

> **输入：** `[3,2,3,1,2,4,5,5,6]`, `k = 4`
> **输出：** 4

提示：

- $1 <= k <= nums.length <= 10^5$
- $-10^4 <= nums[i] <= 10^4$

74. 数组中的第K个最大元素 (No.215) MEDIUM

Solution 1 – 堆排序

思路

本题是堆排序非常经典的模板。最大堆调整max_heapify、创建最大堆build_max_heap和堆排序heapsort的代码应当熟记于心(最小堆同理)。

max_heapify:找到节点i及其孩子中最大的节点，不断下沉调整;

build_max_heap:从第一个非叶子节点往前堆调整;

heapsort:反复把堆顶元素和数组末尾元素交换，然后对前面的新序列进行堆调整。

当本身代码不是很复杂的时候要能全部手写，如果本身题目比较复杂，可以直接调用heapq的heappush和heappop，内部会自动完成堆调整操作，不过注意heapq维护小顶堆，需要大顶堆时需要把相反数入堆，取堆顶元素时再取反。

时间复杂度: $\mathcal{O}(n\log n)$，每次堆调整耗费$\mathcal{O}(\log n)$

空间复杂度: $\mathcal{O}(\log n)$，这是堆调整耗费的递归栈空间

```python
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def max_heapify(heap, i, heap_size):
            # 对节点i进行堆调整
            left, right = 2*i+1, 2*i+2
            largest = i
            # 从根、左孩子、右孩子找最大节点
            if left < heap_size and heap[left] > heap[largest]:
                largest = left
            if right < heap_size and heap[right] > heap[largest]:
                largest = right
            # 最大节点不是根，下沉处理
            if largest != i:
                heap[i], heap[largest] = heap[largest], heap[i]
                max_heapify(heap, largest, heap_size)  # 最大节点递归调整

        def build_max_heap(heap, heap_size):
            # 创建大顶堆，从最后一个非叶子节点开始往前做堆调整
            for i in range(heap_size//2 - 1, -1, -1):
                max_heapify(heap, i, heap_size)

        n = len(nums)
        build_max_heap(nums, n)
        for i in range(n-1, n-k, -1):
            # 堆排序，把最大值移动到最后
            nums[i], nums[0] = nums[0], nums[i]
            # 最大堆调整，后面排好的不调整，要不断缩减heap_size
            max_heapify(nums, 0, i)
        return nums[0]
```

74. 数组中的第K个最大元素 (No.215) MEDIUM

Solution 2 – 快速选择

```python
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def partition(nums, left, right):
            """
            nums[left:right+1]随机选择pivot
            大于pivot的放pivot右边，小于pivot的放pivot左边
            返回pivot在排序后的下标
            也可以选择nums[left]直接做pivot，但容易退化
            """
            i = randint(left, right)  # 随机选pivot下标
            pivot = nums[i]
            # 交换pivot和第一个元素
            nums[i], nums[left] = nums[left], nums[i]
            # 开始处理pivot右侧所有元素
            i, j = left+1, right
            while True:
                # 从左往右找第一个>=pivot的数
                while i <= j and nums[i] < pivot:
                    i += 1
                # 从右往左找第一个<=pivot的数
                while i <= j and nums[j] > pivot:
                    j -= 1
                # 左右指针相遇退出
                if i >= j:
                    break
                # 交换nums[i]和nums[j]，指针往内移
                nums[i], nums[j] = nums[j], nums[i]
                i += 1
                j -= 1
            # nums[j]此时的位置就是pivot的正确排序位置
            # i可能越界，j最坏就是跟left重合
            nums[j], nums[left] = nums[left], nums[j]
            return j
```

思路

快速选择通过快速排序的逻辑查找元素。partition每次随机找一个pivot，≥pivot的元素移到右边，≤pivot的元素移到左边，中间是pivot排序后的正确位置，返回pivot下标。

快速选择的主体只需要不断partition直到返回的下标是n-k。

时间复杂度：$\mathcal{O}(n)$，快速选择的时间是线性的，每次partition需要$\mathcal{O}\left(\frac{n}{2^{m-1}}\right)$，$m$是partition的次数，而$\sum\frac{1}{2^{m-1}}$收敛到常数；

空间复杂度：$\mathcal{O}(1)$

```python
# 续左侧代码
def findKthLargest(self, nums: List[int], k: int) -> int:
    n = len(nums)
    left, right = 0, n-1
    # 快速选择
    while True:  # 找第k大的数保证能找到
        i = partition(nums, left, right)
        if i == n - k:  # 找到了第k大的数
            return nums[i]
        if i > n - k:  # 第k大在pivot左侧
            right = i - 1
        else:  # 第k大在pivot右侧
            left = i + 1
```

74. 数组中的第K个最大元素 (No.215) MEDIUM

补充 – 快速排序

```python
class Solution:
    def sortArray(self, nums: List[int]) -> None:
        def partition(nums, left, right):
            """
            nums[left:right+1]随机选择pivot
            大于pivot的放pivot右边，小于pivot的放pivot左边
            返回pivot在排序后的下标
            也可以选择nums[left]直接做pivot，但容易退化
            """
            i = randint(left, right)  # 随机选pivot下标
            pivot = nums[i]
            # 交换pivot和第一个元素
            nums[i], nums[left] = nums[left], nums[i]
            # 开始处理pivot右侧所有元素
            i, j = left+1, right
            while True:
                # 从左往右找第一个>=pivot的数
                while i <= j and nums[i] < pivot:
                    i += 1
                # 从右往左找第一个<=pivot的数
                while i <= j and nums[j] > pivot:
                    j -= 1
                # 左右指针相遇退出
                if i >= j:
                    break
                # 交换nums[i]和nums[j]，指针往内移
                nums[i], nums[j] = nums[j], nums[i]
                i += 1
                j -= 1
            # nums[j]此时的位置就是pivot的正确排序位置
            # i可能越界，j最坏就是跟left重合
            nums[j], nums[left] = nums[left], nums[j]
            return j
```

思路

这里补充一下快速排序代码，完全复用Solution2中的partition函数，然后需要quickSort递归处理pivot左侧和pivot右侧两部分进行排序。

时间复杂度：$\mathcal{O}(n\log n)$

空间复杂度：$\mathcal{O}(\log n)$，递归栈空间

```python
# 续左侧代码
class Solution:
    def sortArray(self, nums: List[int]) -> None:
        def quickSort(nums, left, right):
            # 对nums[left:right+1]进行快速排序
            if left >= right:
                return
            # i表示pivot的位置
            i = partition(nums, left, right)
            # 递归处理左半部分(left到i-1)
            quickSort(nums, left, i - 1)
            # 递归处理右半部分(i + 1 到 right)
            quickSort(nums, i + 1, right)

        quickSort(nums, 0, len(nums) - 1)
```

75. 前K个高频元素 (No.347) MEDIUM

给你一个整数数组 `nums` 和一个整数 `k` ，请你返回其中出现频率前 `k` 高的元素。你可以按 **任意顺序** 返回答案。

示例 1：

> **输入：** `nums = [1,1,1,2,2,3], k = 2`
>
> **输出：** `[1,2]`

示例 2：

> **输入：** `nums = [1], k = 1`
>
> **输出：** `[1]`

示例 3：

> **输入：** `nums = [1,2,1,2,1,2,3,1,3,2], k = 2`
>
> **输出：** `[1,2]`

提示：

- $1 <= nums.length <= 10^5$
- $-10^4 <= nums[i] <= 10^4$
- `k` 的取值范围是 `[1, 数组中不相同的元素的个数]`
- 题目数据保证答案唯一，换句话说，数组中前 `k` 个高频元素的集合是唯一的

进阶： 你所设计算法的时间复杂度 **必须** 优于 `O(n log n)` ，其中 `n` 是数组大小。

75. 前K个高频元素 (No.347) MEDIUM

Solution 1 – 最小堆

思路

堆排序很经典的解决场景就是找到数组中最大的k个数，维护一个最小堆，堆顶是守门员，比守门员还小那就肯定不是最大的k个数，比守门员大就让它入堆，但是守门员要被淘汰掉。

这题只不过再套个壳，需要统计一下频率，按照频率堆排序而不是数值大小，因此再用一个哈希表存储计数即可。

时间复杂度：$\mathcal{O}(n\log k)$，每次堆调整耗费$\mathcal{O}(\log k)$

空间复杂度：$\mathcal{O}(n)$，因为有哈希表。

```python
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        def min_heapify(heap, i, heap_size):
            # 最小堆调整，排序依据是计数
            left, right = 2*i+1, 2*i+2
            smallest = i
            if left < heap_size and num_map[heap[left]] < num_map[heap[smallest]]:
                smallest = left
            if right < heap_size and num_map[heap[right]] < num_map[heap[smallest]]:
                smallest = right
            if smallest != i:
                heap[i], heap[smallest] = heap[smallest], heap[i]
                min_heapify(heap, smallest, heap_size)

        def build_min_heap(heap, heap_size):
            # 创建最小堆
            for i in range(heap_size//2 - 1, -1, -1):
                min_heapify(heap, i, heap_size)

        n = len(nums)
        # 计数器num_map记录每个元素出现次数
        heap, num_map = [], Counter(nums)
        # 大小为k的最小堆heap，放最大的k个数
        cnt = 0
        for num in num_map:
            if cnt < k:  # stage1：不满k个，入堆阶段
                heap.append(num)
            if cnt == k:  # 刚好k个，创建最小堆
                build_min_heap(heap, k)
            if cnt >= k:  # 超过k个，开启淘汰机制
                # num如果计数比堆顶守门员大，就把堆顶踢出去，比堆顶还小就不予考虑
                if num_map[num] > num_map[heap[0]]:
                    heap[0] = num
                    min_heapify(heap, 0, k)
            cnt += 1
        return heap
```

75. 前K个高频元素 (No.347) MEDIUM

Solution 2 – 桶排序

思路

用桶排序可以实现$\mathcal{O}(n)$的时间复杂度。先对nums计数，然后以计数的最大值作为长度创建一个桶buckets，每个桶放计数相同的所有元素(列表)。由于桶本身下标就是计数，因此下标最大的k个元素即为所求，直接倒序遍历桶即可。

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(n)$，因为桶的长度依赖于最大计数max_cnt，而max_cnt不超过数组长度。

```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        num_map = Counter(nums)
        max_cnt = max(num_map.values())  # max_cnt是最大的计数值
        # buckets的key是计数，value是对应计数的数值列表
        buckets = [[] for _ in range(max_cnt+1)]  # buckets[0]废弃
        # 相同计数加入到一个桶里面
        for num in num_map:
            buckets[num_map[num]].append(num)
        # 倒序前k个就是最大计数
        res, cnt = [], 0
        for bucket in reversed(buckets):
            res.extend(bucket)
            cnt += len(bucket)
            if cnt == k:
                return res
```

76. 数据流的中位数 (No.295) HARD

中位数是有序整数列表中的中间值。如果列表的大小是偶数，则没有中间值，中位数是两个中间值的平均值。

- 例如 `arr = [2,3,4]` 的中位数是 `3` 。
- 例如 `arr = [2,3]` 的中位数是 `(2 + 3) / 2 = 2.5` 。

实现 MedianFinder 类:

- `MedianFinder()` 初始化 `MedianFinder` 对象。
- `void addNum(int num)` 将数据流中的整数 `num` 添加到数据结构中。
- `double findMedian()` 返回到目前为止所有元素的中位数。与实际答案相差 10^{-5} 以内的答案将被接受。

示例 1:

输入
```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
[[], [1], [2], [], [3], []]
```
输出
```
[null, null, null, 1.5, null, 2.0]
```

解释
```
MedianFinder medianFinder = new MedianFinder();
medianFinder.addNum(1);    // arr = [1]
medianFinder.addNum(2);    // arr = [1, 2]
medianFinder.findMedian(); // 返回 1.5 ((1 + 2) / 2)
medianFinder.addNum(3);    // arr[1, 2, 3]
medianFinder.findMedian(); // return 2.0
```

提示:

- $-10^5 <= num <= 10^5$
- 在调用 `findMedian` 之前，数据结构中至少有一个元素
- 最多 $5 * 10^4$ 次调用 `addNum` 和 `findMedian`

76. 数据流的中位数 (No.295) HARD

思路

借鉴[68.寻找两个正序数组的中位数]的思路，找中位数需要维护一个较小组和一个较大组，分别用最大堆left和最小堆right实现，并设置left至多比right多1个元素。这样两堆长度相等时返回堆顶平均值，长度不等时返回left堆顶。

由于heapq不支持最大堆，在left中存储相反数，取数时记得取反。

当加入新数时，如果left和right长度相等，先加入right，弹出的最小值再加入left；如果left更长，先加入left，弹出的最大值加入right。这样能保证新数与已有的数通过堆调整全都能比较一遍。

```python
class MedianFinder:

    def __init__(self):
        self.left = []  # 最大堆，存储较小组
        self.right = []  # 最小堆，存储较大组，存储

    def addNum(self, num: int) -> None:
        # 长度相等时，加入right，然后返回一个最小值加入left
        if len(self.left) == len(self.right):
            heapq.heappush(self.left, -heapq.heappushpop(self.right, num))
        else:  # left比right多1个元素时，加入left，然后返回一个最大值加入right
            heapq.heappush(self.right, -heapq.heappushpop(self.left, -num))

    def findMedian(self) -> float:
        # 长度相等返回堆顶平均值，否则返回left堆顶
        if len(self.left) == len(self.right):
            return (-self.left[0] + self.right[0]) / 2
        return -self.left[0]

# Your MedianFinder object will be instantiated and called as such:
# obj = MedianFinder()
# obj.addNum(num)
# param_2 = obj.findMedian()
```

时间复杂度：findMedian是$\mathcal{O}(1)$，addNum是$\mathcal{O}(\log n)$；**空间复杂度**：$\mathcal{O}(n)$